

Weeks 3-4: Elementary data structures and their processing

Exercise (1) (Week 3)

List Abstract Data Type (ADT) and Linked List. Note that your main code must test all implemented functions.

(A) Implement the List ADT using a linked list that you implement from scratch. Use the code provided in Week 2 as a starting point, and write the main function that tests your implementation. Your linked list should support the following operations and should store a generic element: insertion at the beginning, middle (given a position), and end of the list; deletion of an element from the beginning, middle (given a position), and end of the list; check if a given element exists in the list; and traversal of the list to print all elements.

We take the files `simple_list.h` and `simple_linked_list` from week 2 as a starting point. For this assignment, we made a few specific choices. We used 0 based indexing for `insert_at` and `remove_at`, so `pos = 0` always means the very first element. We added `const` to read-only functions like `contains()` and `print()` to prevent accidental changes to the list.

From `simple_list.h`

```
//new methods
virtual void insert(int pos, const Object x) = 0;

virtual Object remove(int pos) = 0;

virtual bool contains(const Object x) = 0;

virtual void print() = 0;

};
```

From `simple_linked_list.h`

```

//new methods implemented

// Insert
void insert_at(int pos, const Object x) {
    assert(pos >= 0 && pos <= theSize);

    Node* prev = head;
    for (int i = 0; i < pos; i++) {
        prev = prev->next;
    }

    Node* p = new Node;
    p->data = x;
    p->next = prev->next;

    prev->next = p;
    theSize++;
}

// Remove
Object remove_at(int pos) {
    assert(pos >= 0 && pos < theSize);

    Node* prev = head;
    for (int i = 0; i < pos; i++) {
        prev = prev->next;
    }

    Node* to_delete = prev->next;
    Object x = to_delete->data;

    prev->next = to_delete->next;

    delete to_delete;
    theSize--;

    return x;
}

```

```

// Contain
bool contains(const Object x) {
    Node* p = head->next;

    while (p != tail) {
        if (p->data == x) {
            return true;
        }
        p = p->next;
    }

    return false;
}

void print() {
    Node* p = head->next;
    std::cout << "[ ";
    while (p != tail) {
        std::cout << p->data << " ";
        p = p->next;
    }
    std::cout << "]" << std::endl;
}
};

```

From test.cpp:

```
// 1. Test Insertions
list.push_back(10);
list.push_back(30);
cout << "Initial list:          ";
list.print(); // Expected: [ 10 30 ]

list.insert_at(1, 20); // Insert in the middle
cout << "After insert_at(1, 20):  ";
list.print(); // Expected: [ 10 20 30 ]

list.insert_at(0, 5); // Insert at the beginning
cout << "After insert_at(0, 5):    ";
list.print(); // Expected: [ 5 10 20 30 ]

// 2. Test Search
cout << "\n--- CONTAINS TEST ---" << endl;
cout << "Is 20 in the list? " << (list.contains(20) ? "Yes" : "No") << endl; // Expected: Yes
cout << "Is 99 in the list? " << (list.contains(99) ? "Yes" : "No") << endl; // Expected: No

// 3. Test Deletion
cout << "\n--- REMOVE AT TEST ---" << endl;
int removed = list.remove_at(2);
cout << "Removed element at pos 2:  " << removed << endl;
cout << "After remove_at(2):      ";
list.print(); // Expected: [ 5 10 30 ]

cout << "\n--- FINAL STATUS TEST ---" << endl;
cout << "Current size: " << list.size() << endl;
list.clear();
cout << "After clear(), is empty?  " << (list.empty() ? "Yes" : "No") << endl;

return 0;
```

(b) Add, implement, and test, a function to the List ADT to reverse the list

From simple_list.h

```
//new method for exercise 1 b
virtual void reverse() = 0;
```

From simple_linked_list.h

```

void reverse() {
    if (theSize <= 1) return; // Nothing to reverse

    Node* prev = nullptr;
    Node* current = head->next;
    Node* next_node = nullptr;

    // Save reference to the old first element
    Node* old_first = head->next;

    // Flip next pointers for all real nodes
    while (current != tail) {
        next_node = current->next;
        current->next = prev;
        prev = current;
        current = next_node;
    }

    // Fix sentinel connections
    head->next = prev; // head points to original last element
    old_first->next = tail; // original first element now points to tail
}

```

From test.cpp

```

// Test Reversing the List
cout << "\n--- REVERSE TEST ---" << endl;
list.clear();
list.push_back(1);
list.push_back(2);
list.push_back(3);
list.push_back(4);

cout << "Before reverse: ";
list.print(); // Expected: [ 1 2 3 4 ]

list.reverse();

cout << "After reverse: ";
list.print(); // Expected: [ 4 3 2 1 ]

```

Exercise (2)

(Week 3) Implement a Stack using an ordinary array. The stack should be initialized with a size parameter that defines the initial size of the stack. The default constructor should create a stack with 100 elements as the initial size. When push(...) is called and the stack is full, a new array should be allocated that have the double size, the content of the old array copied to the new one and the old array deleted. Analyze the running time of the stack operations push and pop in terms of the N elements it stores

Stack.cpp:

```

template <typename Object>
Stack<Object>::Stack(int initialCapacity) {
    if (initialCapacity <= 0) initialCapacity = 100;
    capacity = initialCapacity;
    array = new Object[capacity];
    topIndex = -1;
}

template <typename Object>
Stack<Object>::~~Stack() {
    delete[] array;
}

template <typename Object>
void Stack<Object>::resize() {
    int newCapacity = capacity * 2;
    Object* newArray = new Object[newCapacity];

    for (int i = 0; i <= topIndex; ++i) {
        newArray[i] = array[i];
    }

    delete[] array;
    array = newArray;
    capacity = newCapacity;
}

template <typename Object>
bool Stack<Object>::empty() const {
    return topIndex == -1;
}

template <typename Object>
int Stack<Object>::size() const {
    return topIndex + 1;
}

```

```

template <typename Object>
int Stack<Object>::size() const {
    return topIndex + 1;
}

template <typename Object>
void Stack<Object>::push(const Object& x) {
    if (topIndex + 1 == capacity) {
        resize();
    }
    array[++topIndex] = x;
}

template <typename Object>
Object Stack<Object>::pop() {
    if (empty()) {
        throw std::underflow_error("Stack is empty");
    }
    return array[topIndex--];
}

template <typename Object>
Object Stack<Object>::top() const {
    if (empty()) {
        throw std::underflow_error("Stack is empty");
    }
    return array[topIndex];
}

```

Push(): normally runs $O(1)$ constant time but in worst case it runs $O(n)$, because all the N s must be copied when resizing

Pop(): decrements the counter then returns the top element so run time is $O(1)$

Exercise (3)

Week 3) Write an implementation of Queue using Stack(s). The implementation must be a template and use the Stack ADT presented in the course. There is no requirement that the implementation is efficient, but you cannot use the internal structure of the stack, nor an iterator for the stack.

From queue_from_stacks.cpp:

```

template <typename Object>
class Queue {
private:
    Stack<Object> inStack;
    Stack<Object> outStack;

    // Helper method to transfer elements from inStack to outStack
    void shiftStacks() {
        if (outStack.empty()) {
            while (!inStack.empty()) {
                outStack.push(inStack.pop());
            }
        }
    }

public:
    Queue() = default;
    ~Queue() = default;

    bool empty() const {
        return inStack.empty() && outStack.empty();
    }

    int size() const {
        return inStack.size() + outStack.size();
    }

    void enqueue(const Object& x) {
        inStack.push(x);
    }

```

```

Object dequeue() {
    if (empty()) {
        throw std::underflow_error("Queue is empty");
    }
    shiftStacks();
    return outStack.pop();
}

Object front() {
    if (empty()) {
        throw std::underflow_error("Queue is empty");
    }
    shiftStacks();
    return outStack.top();
}

```

(4) (Week 4) Let T be a hash-table of size 7 with the hash function $h(x) = x \bmod 7$. Write down the entries of T after the keys 5, 28, 19, 15, 20, 33, 12, 17, 33 and 10 have been inserted using

- a) chaining
- b) linear probing
- c) quadratic probing.

What is the load-factor(λ) in the three case

Hash Table Size (m): 7

Hash Function: $h(x) = x \bmod 7$

Keys to insert: 5, 28, 19, 15, 20, 33, 12, 17, 33, 10 (10 insertions total)

Initial Hash Values

- $h(5) = 5 \bmod 7 = 5$
- $h(28) = 28 \bmod 7 = 0$
- $h(19) = 19 \bmod 7 = 5$
- $h(15) = 15 \bmod 7 = 1$
- $h(20) = 20 \bmod 7 = 6$

- $h(33) = 33 \bmod 7 = 5$
- $h(12) = 12 \bmod 7 = 5$
- $h(17) = 17 \bmod 7 = 3$
- $h(33) = 33 \bmod 7 = 5$
- $h(10) = 10 \bmod 7 = 3$

a) Chaining

Index	Stored Key
0	28
1	15
2	<i>Empty</i>
3	17 -> 10
4	<i>Empty</i>
5	5 -> 19 -> 33 -> 12 -> 33
6	20

Load Factor: $10/7 = 1.43$

b) Linear probing

If there is no space in its index it will be moved to the next one, also with linear probing there is only space for exactly 7 keys if there is 7 buckets.

Index	Stored Key
0	28
1	15
2	20
3	33
4	12
5	5
6	19

Load Factor: $7/7 = 1.0$

c) Quadratic probing

Index	Stored Key
0	28

1	15
2	33
3	20
4	17
5	5
6	19

Load Factor: $7/7 = 1.0$

(5) (Week 4) Implement a Set using either a Queue, List or Stack ADT as presented in the course. There is no requirement that the implementation is efficient, but you cannot use the internal structure of the ADT, nor an iterator for the ADT

We chose to use a queue. We insert duplicate keys as separate entries. Also It stops accepting insertions after 7 successful writes and marks remaining keys as failed.

```
// Checks if an element exists in the set
bool contains(const T& element) {
    bool found = false;
    size_t n = q.size();

    // Full rotation to inspect each element while preserving original order
    for (size_t i = 0; i < n; ++i) {
        T item = q.front();
        q.pop();

        if (item == element) {
            found = true;
        }

        q.push(item); // Rotate back to rear
    }

    return found;
}

// Inserts an element only if it is not already present
bool add(const T& element) {
    if (!contains(element)) {
        q.push(element);
        return true;
    }
    return false;
}
```

```

// Removes an element from the set if present
bool remove(const T& element) {
    if (!contains(element)) {
        return false;
    }

    size_t n = q.size();
    for (size_t i = 0; i < n; ++i) {
        T item = q.front();
        q.pop();

        // Re-enqueue everything EXCEPT the target element
        if (item != element) {
            q.push(item);
        }
    }

    return true;
}

// Returns current size of the set
size_t size() const {
    return q.size();
}

// Checks if the set is empty
bool is_empty() const {
    return q.empty();
}

```

```

int main() {
    QueueSet<int> setA;

    // Test add and duplicate prevention
    setA.add(10);
    setA.add(20);
    setA.add(30);
    setA.add(20); // Duplicate, will not be added

    std::cout << "Set A: ";
    setA.print(); // Output: { 10, 20, 30 }

    // Test removal
    setA.remove(20);
    std::cout << "Set A after removing 20: ";
    setA.print(); // Output: { 10, 30 }

    // Test union
    QueueSet<int> setB;
    setB.add(30);
    setB.add(40);
    setB.add(50);

    QueueSet<int> setC = setA.set_union(setB);
    std::cout << "Union of A and B: ";
    setC.print(); // Output: { 10, 30, 40, 50 }

    return 0;
}

```

(Week 4) Implement a Dictionary or Map using an STL vector. The element of the vector (for the implementation) must be an STL pair representing the (key, value)

Insertions are made via `push_back()` rather than keeping the vector sorted. Calling `insert()` with an existing key updates its stored value rather than creating duplicate key entries.

```
// Removes a key-value pair if present
bool remove(const K& key) {
    int idx = findIndex(key);
    if (idx != -1) {
        // Erase the element at position idx
        data.erase(data.begin() + idx);
        return true;
    }
    return false;
}

// Returns number of key-value pairs
size_t size() const {
    return data.size();
}

// Checks if map is empty
bool is_empty() const {
    return data.empty();
}

// Overloaded operator[] for convenient key lookup / assignment
V& operator[](const K& key) {
    int idx = findIndex(key);
    if (idx != -1) {
        return data[idx].second;
    }
    // If key doesn't exist, insert default Value and return reference to it
    data.push_back(std::make_pair(key, V()));
    return data.back().second;
}
```

```
VectorMap() = default;

// Inserts a key-value pair. Updates value if key already exists.
void insert(const K& key, const V& value) {
    int idx = findIndex(key);
    if (idx != -1) {
        data[idx].second = value; // Update existing key
    }
    else {
        data.push_back(std::make_pair(key, value)); // Insert new pair
    }
}

// Retrieves value by key. Throws exception if key doesn't exist.
V get(const K& key) const {
    int idx = findIndex(key);
    if (idx != -1) {
        return data[idx].second;
    }
    throw std::out_of_range("Key not found in map.");
}

// Checks if key exists in the map
bool contains(const K& key) const {
    return findIndex(key) != -1;
}
```

```

VectorMap<std::string, int> ageMap;

// Insert key-value pairs
ageMap.insert("Alice", 25);
ageMap.insert("Bob", 30);
ageMap.insert("Charlie", 22);

std::cout << "Initial Map: ";
ageMap.print();

// Update value for existing key
ageMap.insert("Alice", 26);
std::cout << "After updating Alice's age: ";
ageMap.print();

// Look up value
if (ageMap.contains("Bob")) {
    std::cout << "Bob's age: " << ageMap.get("Bob") << "\n";
}

// Subscript operator [] lookup and assignment
ageMap["Diana"] = 28; // Inserts new entry
std::cout << "After adding Diana with []: ";
ageMap.print();

// Remove key
ageMap.remove("Charlie");
std::cout << "After removing Charlie: ";
ageMap.print();

return 0;

```

7) (Week 4) An old exam question

Nedenstående tabel er en hashtable, hvor der anvendes quadratic probing i tilfælde af kollisioner (collision resolution). Hash funktionen er $h(x) = x \bmod 11$. Følgende værdier er indsat: 22, 5, 16 og 27 (i denne rækkefølge):

Index	Value
0	22
1	
2	
3	
4	
5	5
6	16
7	
8	
9	27
10	

Positionerne (indexes) 1, 2, 3, 4, 7, 8 og 10 er ubrugte. Verificer at hashtabellen er korrekt og ret eventuelle fejl. Vis tabellens udseende efter at elementerne 1, 12, din alder samt dit studienummer er indsat og begrund hvorfor det indsættes hvor det indsættes samt forklar hvad det derudover sker. Noter i besvarelsen hvad din alder og studienummer er.

We verify the hashtable:

22 mod 11 is 0.

5 mod 11 is 5

16 mod 11 is 5 so we use quadratic probing by saying $5 + 1^2$ which is 6.

27 mod 11 is 5. $5 + 1^2$ is 6. $5 + 2^2$ is 9.

So the hashtable is correct

We now insert 1, 2, my age (22) and my studienummer (802717)

1 mod 11 is 1.

12 mod 11 is 1. $1 + 1^2 = 2$

22 mod 11 is 0. $0 + 1^2$ is 1. $0 + 2^2$ is 4.

802717 mod 11 is 3.

We now fill up the table

Index	Key
0	22
1	1
2	12
3	802717
4	22
5	5
6	16
7	
8	
9	27

10	
----	--